

RIDE TEA GUPSHUP – IMPLEMENTATION PLAN

Phase 2: Admin Login & Database

Supabase-backed content management for the RTG website

Generated 2026-08-06

Phase 2 — Admin Login + Supabase Database for RTG

Context

The RTG site (React 19 + Vite + Tailwind v4 + Framer Motion + React Router) is currently a pure static SPA — every event, product, blog post, testimonial, sponsor, and image lives in two hardcoded files (`src/data/content.js` , `src/data/images.js`). The owner has no way to change anything without a developer editing code and redeploying.

Phase 2 fixes that: an admin-only login, a real database, and an admin dashboard so the owner can add events, upload gallery/product photos, and update site content themselves, from a phone or laptop browser — with changes appearing on the live site immediately.

Decided with user: Supabase (Postgres + Auth + Storage) · admin/owner login only (no public member accounts yet) · this phase covers content management only (contact form / newsletter wiring is a later phase).

Architecture

Supabase (one project, free tier to start) gives us three things with no server to run:

- **Postgres database** — relational tables for events, gallery, products, blog, sponsors, testimonials, challenges, and site-wide images
- **Auth** — email/password login for admin accounts, no public signup UI
- **Storage** — a public bucket for uploaded photos/videos, with authenticated-only writes

The React app talks to Supabase directly via `@supabase/supabase-js` — no custom backend to build or host.

Database Schema

```
-- Optional: display name for logged-in admins
create table admin_profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text,
  created_at timestamptz default now()
);

create table events (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null,
  title text not null,
  event_date text not null,      -- display string, e.g. "June 2027" (matches current
  event_type text,
  categories text[] default '{}',
  prize_pool text,
  description text,
  cover_image_url text,
  featured boolean default false,
  sort_order int default 0,
  published boolean default true,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

create table gallery_items (
  id uuid primary key default gen_random_uuid(),
  media_type text not null check (media_type in ('image','video')),
  media_url text not null,
  caption text,
  sort_order int default 0,
  published boolean default true,
  created_at timestamptz default now()
);

create table products (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  price numeric(10,2) not null,
  image_url text,
  tag text,
  in_stock boolean default true,
  sort_order int default 0,
  created_at timestamptz default now()
);

create table blog_posts (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null,
  title text not null,
  category text,
```

```

excerpt text,
content text,
cover_image_url text,
published boolean default true,
published_at timestampz default now(),
created_at timestampz default now()
);

create table sponsors (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  logo_url text,
  tier text,
  website_url text,
  sort_order int default 0,
  created_at timestampz default now()
);

create table testimonials (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  role text,
  quote text not null,
  avatar_url text,
  sort_order int default 0,
  published boolean default true,
  created_at timestampz default now()
);

create table challenges (
  id uuid primary key default gen_random_uuid(),
  title text not null,
  period text,
  description text,
  sort_order int default 0,
  created_at timestampz default now()
);

-- Every sitewide image (hero banners, etc.) as one editable key -> url table,
-- mirroring the keys already in src/data/images.js exactly.
create table site_images (
  key text primary key,
  url text not null,
  label text,          -- human-friendly name shown in the admin UI, e.g. "Home Page H
  updated_at timestampz default now()
);

```

Row Level Security — same pattern on every content table (public can read, only logged-in admins can write):

```

alter table events enable row level security;
create policy "public read" on events for select using (published = true);
create policy "admin read all" on events for select using (auth.role() = 'authenticated');
create policy "admin write" on events for insert with check (auth.role() = 'authenticated');
create policy "admin update" on events for update using (auth.role() = 'authenticated');
create policy "admin delete" on events for delete using (auth.role() = 'authenticated');

```

`auth.role() = 'authenticated'` is safe here because **no public signup exists** — the only way to hold a valid session is to be an admin we created manually. Repeat this policy block per table (products/sponsors/challenges/site_images skip the `published` split and just allow public select on everything, since they have no `published` column).

Storage: one public bucket `rtg-media` — public read (default for a public bucket), insert/update/delete restricted to `auth.role() = 'authenticated'`.

Frontend Implementation

New shared data layer

- `src/lib/supabaseClient.js` — client from `import.meta.env.VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY`
- `src/lib/publicData.js` — read hooks for public pages: `useEvents()`, `useGalleryItems()`, `useProducts()`, `useBlogPosts()`, `useSponsors()`, `useTestimonials()`, `useChallenges()`, `useSiteImages()`. Each hook **initializes its state from the existing static `content.js` / `images.js` values**, then fetches from Supabase and swaps in live data when it arrives — so the site never shows a blank/loading flash, and still works even if Supabase is briefly unreachable.

New `src/admin/` area

- `AuthContext.jsx` — wraps `supabase.auth.getSession()` + `onAuthStateChange`, exposes `{ user, loading, signIn, signOut }`
- `ProtectedRoute.jsx` — redirects to `/admin/login` if no session
- `AdminLayout.jsx` — sidebar nav (Events, Gallery, Products, Blog, Sponsors, Testimonials, Challenges, Site Images) + logout button
- `useTable.js` — generic hook wrapping select/insert/update/delete for a given table name
- `resourceConfig.js` — declares field schema per entity (text / number / textarea / image / tags / boolean / select) — this one config file drives generic list+form UI reused for **Events, Products, Blog, Sponsors, Testimonials, Challenges** (6 structurally-identical CRUD screens), avoiding six near-duplicate page implementations
- `components/ResourceTable.jsx`, `components/ResourceForm.jsx` — generic, config-driven
- `components/ImageUploadField.jsx` — file picker → uploads to the `rtg-media` bucket → returns a public URL → shown as a preview; used inside every form
- `pages/Login.jsx`, `pages/Dashboard.jsx` (simple overview/counts)

- `pages/GalleryAdmin.jsx` — bespoke (media grid + multi-file upload, image or video), doesn't fit the generic "row" pattern
- `pages/SiteImagesAdmin.jsx` — bespoke: fixed list of every `site_images` key with its current thumbnail + a "Replace" button — this is the screen that lets the owner swap the homepage hero, about page hero, etc.

Routing (`App.jsx`)

- New top-level route group for `/admin/*`, using `AdminLayout` instead of the public `Layout` (no navbar/footer) — `/admin/login` public, everything else under `ProtectedRoute`
- `Preloader` (video intro) skipped on `/admin/*` paths

Migrating existing pages

- `Home.jsx`, `Events.jsx`, `Merchandise.jsx`, `Blog.jsx`, `Sponsors.jsx`, `Gallery.jsx`, `Challenges.jsx` swap their static `content.js` / `images.js` imports for the corresponding `publicData.js` hooks. `content.js` / `images.js` themselves stay in the repo as the fallback/default values these hooks start from — nothing about the current visual design changes.

Setup files

- `supabase/schema.sql` — full migration: all `CREATE TABLE`, RLS policies, bucket setup, **plus seed `INSERT` statements populated from the current `content.js/images.js`** so the database starts pre-loaded with the real current site content, not empty
- `.env.example` committed; `.env.local` gitignored (will confirm `.gitignore` covers this)
- `package.json` — add `@supabase/supabase-js`

Implementation Sequence

1. `supabase/schema.sql` (tables, RLS, storage bucket + policies, seed data) + `.env.example`
2. `src/lib/supabaseClient.js` + `AuthContext` + `ProtectedRoute` + `Login.jsx` — prove login works end-to-end
3. Generic CRUD layer (`useTable`, `resourceConfig`, `ResourceTable`, `ResourceForm`, `ImageUploadField`) + `AdminLayout`
4. Wire the 6 generic admin pages (Events, Products, Blog, Sponsors, Testimonials, Challenges)
5. Bespoke `GalleryAdmin.jsx` and `SiteImagesAdmin.jsx`
6. `publicData.js` hooks + migrate the 7 public pages to read from them
7. Update `App.jsx` routing + README/setup docs

How the owner will use it day-to-day

1. Go to `yoursite.com/admin/login`, sign in with the email/password we set up for them (created once via the Supabase dashboard — no signup page exists)
2. Land on a dashboard with a sidebar: Events, Gallery, Products, Blog, Sponsors, Testimonials, Challenges, Site Photos

3. To add an event: **Events** → **Add Event**, fill in title/date/description, click the photo field to upload a cover image from their phone/computer, **Save** — it appears on the public Events page immediately
4. To add gallery photos/videos: **Gallery** → **Upload**, pick one or more files
5. To change any sitewide banner (home hero, about hero, etc.): **Site Photos**, find the labeled slot (e.g. "Home Page Hero"), click **Replace**, pick a new file
6. No code, no file paths, no developer needed for routine updates — works from any browser, including mobile

Risks / Caveats to flag to the user

- **Video storage on the free tier:** Supabase's free plan has limited storage/bandwidth (~1GB storage / a few GB egress). Photos are fine; gallery *videos* can eat this quickly. Recommend keeping uploaded videos short and compressed, and note that migrating heavy video to YouTube/Vimeo embeds (paste a link instead of uploading the file) is a clean upgrade path if this becomes a problem.
- **First admin account is manual:** since there's no signup page, the first login is created once via the Supabase dashboard (Authentication → Add user) — a one-time setup step, not code.
- **Supabase project creation itself is a manual step:** I can write all the code and the schema, but creating the Supabase account/project and pasting the resulting URL/key into `.env.local` has to be done by the user (or the owner) since it requires a browser + account signup I can't do on their behalf.
- **No image optimization/resizing yet:** uploaded photos are stored as-is; if the owner uploads huge phone photos (10-20MB), page load could slow down. Worth a follow-up phase (Supabase image transforms or client-side compression before upload) but out of scope for now.

Verification

- `npm run build` succeeds with the new Supabase dependency
- Manual pass: log in at `/admin/login`, add a test event with an uploaded image, confirm it appears on the public `/events` page without a reload issue; edit a `site_images` entry and confirm the corresponding public page picks it up
- Confirm public (logged-out) requests can read but not write (test via browser dev tools / a quick unauthenticated fetch) to sanity-check RLS
- Re-run the same no-console-errors pass used in Phase 1 (Playwright, desktop + mobile) across public pages plus `/admin/login` and one protected admin page